RESEARCH MEMORANDUM

Django vs. Flask for the Sleep Study Recruitment Website

Isha Hebbar

2026-10-26

Table of contents

1 Introduction **1**

2 Key Analysis **2**
2.1 SWOT: Django 2
2.2 SWOT: Flask 3
2.3 SWOT: No-Code Option 4
2.4 Summary Comparison 4

3 Suggestions and Questions **5**

References **5**

1 Introduction

Researcher: Isha Hebbar
Question: Should the sleep study recruitment website be built with Django or Flask?

Recommendation: Django. It comes with login, an RA admin page, and security protections already built in, which fits this project best. A no-code option (React front end plus a university form) is the main alternative.

Constraints considered:

- The grad student knows Python for data analysis but has never built a website.
- The site stores contact info, so security and privacy matter.
- It should be running in about a month.
- Nobody will maintain it full time.

2 Key Analysis

2.1 SWOT: Django

Strengths

- **Login and admin page are built in.** RAs get a ready-made page to see sign-ups, so almost nothing has to be built by hand.[1]
- **Security is on by default.** The site stores names and emails of students, so a leak would be a real privacy problem, and Django blocks the common attacks without extra setup (Django Software Foundation 2025).[2]
- **Fewer architecture choices.** Django decides how the project is organized, so a non-technical builder does not have to pick tools or layouts, though they will still need to learn some of Django's own structure.

Weaknesses

- **Steeper learning curve.** There are many new concepts to learn at once, which is hard for someone who has only done data analysis.
- **Heavy for a small site.** A site with about four pages still comes with many files and folders.
- **Hard to debug.** Django does a lot automatically, so when something breaks it is not obvious why.

Opportunities

- **Easy to reuse for later studies.** A new study would only need a new "model" in the same project.[3]
- **RAs can filter sign-ups in the admin page.** Filtering means showing only some rows, such as sign-ups from this week or people who qualified. Without it, RAs would scroll through the full list or someone would have to build a custom page.
- **Python skills carry over.** Django reads and writes the database with Python code, which feels like working with a data table.[4]

[1] The admin page is an automatic back-office screen where logged-in staff can view and edit the stored data, like a spreadsheet inside the website.

[2] Three common attacks Django blocks automatically. *CSRF*: another website tricks a logged-in RA's browser into submitting actions they never meant to, such as deleting sign-ups. *SQL injection*: someone types code into a form field (like the screener) to read or wipe the database. *XSS*: someone sneaks a malicious script into a form answer that then runs in an RA's browser. Django also stores passwords as scrambled "hashes" so a stolen database does not reveal them. This matters here because the screener collects health-related answers.

[3] A model is a description of one kind of record the site stores. For example, a "SignUp" model says each sign-up has a name, an email, and a date. Django turns it into a database table and an admin page automatically.

[4] This part of Django is called the ORM. It lets you write Python instead of database language, so the grad student does not need to learn a second language.

Threats

- **Falls behind on updates.** Django releases security fixes regularly, and if nobody installs them the site slowly becomes easier to break into.
- **Hosting may delay the timeline.** Campus IT has to support a full Django setup, which could push past the one-month goal.
- **Hard to hand off.** If the grad student leaves, the next person has to learn Django from scratch.

2.2 SWOT: Flask

Strengths

- **Easy to start.** A working page takes about five lines of Python, so the grad student sees results on day one.
- **Small and readable.** The whole site fits in a few files, which makes it easy for a beginner to understand and for a future person to pick up.
- **Screener logic is plain Python.** The grad student can write the eligibility rules in a language they already know.[5]

Weaknesses

- **Security is your job.** Flask has no built-in login or CSRF protection, so extensions like Flask-Login and Flask-WTF must be added and set up correctly (Pallets 2025). For a site holding student contact info, that is a lot of responsibility for a beginner.
- **Many small choices.** Which database tool? Which login add-on? Each choice is a decision a first-time builder is not equipped to make.
- **RA sign-up list is built by hand.** There is no free admin page, so this part adds time to a one-month schedule.

Opportunities

- **Fast prototype.** The PI could see a working draft in the first week or two and give feedback early.
- **Can grow gradually.** If the lab only ever needs small features, the site never becomes more complicated than it has to be.
- **Free to use what IT supports.** Because Flask does not force a setup, it can be fitted to whatever hosting and database campus IT already offers.

Threats

[5] Screener logic is the set of rules that decides who qualifies, such as "age 18 to 25 and sleeps fewer than 6 hours" means eligible. It is the core of this site.

- **One missed setup step can leak data.** A forgotten protection exposes participants' emails, which campus IT will care about.
- **Add-ons can go stale.** They are maintained by different people, so one may stop getting updates while the site still relies on it.
- **No standard layout.** A future maintainer may not understand how the site was put together.

2.3 SWOT: No-Code Option

This option is a basic React front end that describes the study and links to a university-managed form system for the screener and sign-up, with login through Microsoft, which UW already uses. React is a free tool for building simple web pages (Meta Open Source 2025), Microsoft Forms collects the responses (Microsoft 2025b), and Microsoft sign-in controls which RAs can see them (Microsoft 2025a).

- **Strength:** live within days, and the participant data stays in a system IT has already approved.
- **Weakness:** the screener rules and page design are limited to what the form tool allows.
- **Opportunity:** the lab could start with this and move to Django later if the study outgrows it.
- **Threat:** access to responses depends on who the form is shared with, so an RA who leaves could keep access by mistake.

2.4 Summary Comparison

Table 1: Summary comparison for the sleep study site

Question	Django	Flask	No-code
Learning curve for a beginner	Steeper	Gentle	Gentle
RA login + sign-up list	Mostly built in	Build it yourself	Microsoft login + form results
Security by default	Strong	Add-ons needed	Handled by the university
Custom screener logic	Yes	Yes	Limited
Easy to reuse and hand off	Great	Depends	Easy, but less flexible

3 Suggestions and Questions

Suggestion: Have one person learn Django for a few weeks before building, using a Udemy course and the official Django documentation, with extra focus on security and on using the admin page for the RA view. If IT can only host something very simple, use the no-code option or Flask as the fallback.

Questions for IT:

1. Can the university host a Python app with a database, and who would apply security updates after the study starts?
2. Would a university-approved form plus a simple page be acceptable instead of a custom site?
3. Does the IRB need to approve storing screener answers (sleep and health info)?

References

Django Software Foundation. 2025. *Django Documentation: Security in Django*. https://docs.djangoproject.com/en/stable/topics/security/.

Meta Open Source. 2025. *React: Learn React*. https://react.dev/learn.

Microsoft. 2025a. *Microsoft Entra ID Documentation*. https://learn.microsoft.com/en-us/entra/identity/.

Microsoft. 2025b. *Microsoft Forms Help & Learning*. https://support.microsoft.com/en-us/forms.

Pallets. 2025. *Flask Documentation: Security Considerations*. https://flask.palletsprojects.com/en/stable/web-security/.